# TOPOspace

## AI Execution Optimization Plane

A Topology-Aware Middleware for Context, Model, Execution Path, and Token Optimization

**Whitepaper · Commercialization Edition**

Based on TopoSpace Cloud 1.3.0-beta1

**Caleb Lee**
director@j1729labs.online

August 2026

**Abstract**

TopoSpace is a provider-neutral AI execution optimization layer that jointly determines context geometry, model selection, execution path, and token budget for AI agents. It is designed as middleware rather than as a new foundation model or an application framework. The commercial architecture combines a topology-aware optimization runtime, learned Meta-Policy, policy operations, an OpenAI-style cloud gateway, provider routing, resilience controls, usage metering, billing primitives, and graph-based observability.

# Contents

1. Executive Summary
2. Problem Statement
3. Design Principles
4. Mathematical Foundation
5. Topology-Aware Context Runtime
6. Joint Execution Optimization
7. Meta-Policy Control Plane
8. Safe Policy Operations
9. TopoSpace Cloud Architecture
10. Provider Execution and Resilience
11. Graph Visualization and Observability
12. Token and Cost Optimization
13. API and Integration Model
14. Commercial Deployment on Replit
15. Security and Multi-Tenancy
16. Validation Methodology
17. Commercial Product Model
18. Roadmap
19. Limitations and Production Gaps
20. Conclusion

# 1. Executive Summary

Modern AI applications increasingly combine multiple models, tools, memories, agents, and retrieval sources. The resulting execution problem is no longer simply “which LLM should answer?” A production system must decide what context should be visible, which model should execute, which reasoning/tool path should be used, and how much token budget should be allocated - while preserving quality, latency, reliability, policy constraints, and cost.

TopoSpace addresses this as an execution optimization problem. Its core idea is to represent AI objects and context relationships in a task-conditioned topology, select a bounded neighborhood relevant to the current objective, and jointly optimize geometry, model routing, and execution graph through a Meta-Policy control plane.

**Product Definition**
TopoSpace is an AI Execution Optimization Plane: a framework-neutral middleware layer placed between an application/agent and model or tool providers.

# 2. Problem Statement

- Static model selection wastes cost on simple tasks and underperforms on difficult tasks.
- RAG and agent systems often over-include context, increasing token usage and latency.
- Tool, memory, skill, and agent selection becomes combinatorial as systems scale.
- Independent optimization of context, model, and execution path can create locally efficient but globally poor decisions.
- Learned routing policies require shadow evaluation, staged promotion, and rollback before they can safely control production traffic.
- Commercial deployment additionally requires authentication, tenant isolation, quota, rate limiting, usage accounting, provider fallback, and observability.

# 3. Design Principles

| Principle | Meaning |
|---|---|
| Provider-neutral | The optimization plane is separated from model-provider execution. |
| Framework-neutral | TopoSpace can sit behind custom agents or framework adapters rather than depending on one agent framework. |
| Task-conditioned | Geometry and routing respond to task state, risk, ambiguity, hierarchy, and pressure variables. |
| Bounded context | The runtime avoids whole-space context exposure and selects a relevant neighborhood. |
| Joint optimization | Geometry, model, and execution path are coordinated through one Meta-Policy. |
| Safety first | Deterministic policy/security overrides remain outside learned utility. |
| Measurable | Token, cost, latency, success, and policy outcomes are observable and testable. |
| Reversible | Champion/challenger, staged rollout, and rollback protect production traffic. |

# 4. Mathematical Foundation

The original TopoSpace geometry is based on a task-conditioned composite Lp distance. A generalized form is:

$$D(x,y \mid s,g) = \left( \Sigma_j \, w_j(s,g) \cdot D_j(x,y)^{p(s,g)} \right)^{1/p(s,g)}$$

Here, s denotes runtime state and g denotes the current objective. Component distances may encode semantic, structural, dependency, risk, collaboration, memory, or other relationships. The policy can adapt p, component weights, and neighborhood radius ε.

### 4.1 Why generalized distance matters

A single fixed cosine or Euclidean distance is often insufficient for heterogeneous agent systems. TopoSpace treats relevance as a multi-component, task-dependent relation. This makes the geometry itself part of execution policy rather than a static preprocessing choice.

### 4.2 Neighborhood selection

The runtime uses candidate generation followed by geometry-aware ranking and an epsilon or budget constraint. This bounds the visible context and allows the execution layer to reason over a smaller, task-relevant subspace.

## 5. Topology-Aware Context Runtime

TopoSpace can model memory, tools, skills, agents, code, models, and contextual objects as heterogeneous nodes. Edges represent relationships such as semantic similarity, dependency, execution flow, collaboration, or structural proximity.

| Object | Typical role |
|---|---|
| Memory | Recallable historical or durable context |
| Tool | Callable external or internal capability |
| Skill | Reusable execution capability or environment |
| Agent | Specialized reasoning/execution participant |
| Code | Repository/function/module context |
| Model | Candidate inference engine |
| Context | Task/document/state object |

The graph is therefore not only a visualization artifact. It is an execution substrate from which relevant neighborhoods and paths can be selected.

## 6. Joint Execution Optimization

### 6.1 Geometry Policy

Selects a geometry family and associated parameters from runtime state. This controls how context and candidate objects are related.

### 6.2 Model Routing Policy

Selects among model profiles such as fast, balanced, or reasoning-oriented execution based on workload characteristics and learned outcomes.

### 6.3 Execution Graph Policy

Selects a path template such as fast-only, fast-to-tool, memory-to-balanced, balanced-to-tool, balanced-to-reasoning, or memory-to-balanced-to-reasoning.

### 6.4 Token Budget

Token budget is treated as a control variable informed by the selected context, model, path, and task pressure rather than as an isolated hard-coded constant.

## 7. Meta-Policy Control Plane

The Meta-Policy coordinates geometry, model, and execution-path decisions. Its reference utility combines quality, success, cost efficiency, latency efficiency, violations, and instability. The exact coefficients are implementation parameters, not universal constants.

$$U = \alpha Q + \beta S + \gamma C + \delta L - \lambda V - \mu I$$

This formulation is intentionally multi-objective. A routing policy that saves tokens but materially reduces success should not be promoted merely because it is cheaper.

### 7.1 Deterministic overrides

High-risk conditions can force safer model/path choices even when the learned utility prefers a cheaper option. This separates policy enforcement from optimization learning.

# 8. Safe Policy Operations

| Mechanism | Purpose |
|---|---|
| Champion | Current production policy |
| Challenger | Candidate policy evaluated against champion |
| Shadow evaluation | Measures challenger without controlling live output |
| Promotion gate | Requires minimum evidence and guardrail compliance |
| Staged rollout | Expands traffic gradually |
| Rollback | Returns to prior policy after regression |
| Drift monitoring | Detects changes in workload/topology behavior |
| Replay/explain | Supports debugging and auditability |

This operational layer is essential because optimization policies are themselves production software. They must be versioned, evaluated, promoted, and reversible.

# 9. TopoSpace Cloud Architecture

```
Client / Agent
   |
   v
POST /v1/chat/completions
   |
Auth - Tenant - Rate Limit - Quota
   |
Agent Runtime Gateway
   |
Meta-Policy
 |-- Geometry
 |-- Model
 `-- Execution Path
   |
Provider Router
 |-- Primary
 `-- Fallback
   |
Usage - Billing - Telemetry - PolicyOps
```

TopoSpace Cloud 1.3.0-beta1 exposes an ASGI service with a health endpoint and an OpenAI-style chat request surface. The cloud layer converts requests into TopoSpace runtime decisions and, when configured, executes them through an OpenAI-compatible provider adapter.

# 10. Provider Execution and Resilience

| Capability | Current design |
|---|---|
| Provider contract | ProviderAdapter abstraction |
| Real HTTP execution | OpenAI-compatible /chat/completions adapter |
| Retry | Configurable retry count |
| Fallback | Secondary provider after primary failure |
| Circuit breaker | Temporarily isolates repeatedly failing provider |
| Streaming | SSE surface and provider stream contract |
| Metering | Records provider-reported token usage when available |
| Billing | Reference ledger for tenant/request/provider/model usage |

**Production note**
Provider-specific upstream streaming semantics, tool-call normalization, and high-volume failover behavior require provider-specific hardening before full GA traffic.

# 11. Graph Visualization and Observability

TopoSpace uses graph visualization as a runtime debugging and explainability surface. Operators should be able to see not only metrics but also the selected topology and execution path.

| Visual layer | Operational meaning |
|---|---|
| Nodes | Memory/tool/skill/agent/code/model/context |
| Edges | Semantic/structural/dependency/execution relationships |
| Neighborhood | Objects exposed for the current task |
| Execution path | Actual selected graph path |
| Policy overlay | Geometry/model/path decision and override |
| Timeline | State and graph mutation replay |
| Metrics overlay | Tokens, cost, latency, success, drift |

# 12. Token and Cost Optimization

TopoSpace does not define token optimization as simply reducing max_tokens. The intended optimization stack combines context pruning, smaller relevant neighborhoods, model selection, shorter execution graphs, fewer unnecessary tool calls, and task-conditioned token budgets.

## 12.1 Marketing claim discipline

Commercial savings claims must be derived from actual provider-reported usage on representative workloads. Synthetic and deterministic benchmarks in the repository are regression evidence and must not be presented as customer-facing savings claims.

| Claim | Required evidence |
|---|---|
| Token saving | Paired baseline vs TopoSpace provider-reported token usage |
| Cost saving | Documented provider pricing or billed cost |
| Latency | p50/p95 with sample size and workload definition |
| Quality | Success/evaluator non-inferiority or defined acceptable delta |
| Reliability | Error, fallback, and rollback rates |

# 13. API and Integration Model

The primary developer experience is an OpenAI-style HTTP request with optional TopoSpace state extensions.

```
POST /v1/chat/completions
Authorization: Bearer <TOPOSPACE_API_KEY>
Content-Type: application/json

{
  "model": "auto",
  "messages": [{"role":"user","content":"..."}],
  "max_tokens": 800,
  "stream": false,
  "topospace_state": {
    "risk": 0.2,
    "ambiguity": 0.4,
    "hierarchy": 0.5,
    "candidate_pressure": 0.3,
    "latency_pressure": 0.4
  }
}
```

This allows TopoSpace to be offered as middleware without requiring customers to adopt a specific agent framework. Framework adapters can be added where useful.

# 14. Commercial Deployment on Replit

The beta1 package includes `main.py`, `.replit`, `requirements.txt`, `.env.example`, `/healthz`, and an environment-driven bootstrap path. A commercial Replit deployment should separate development, staging, and production.

| Environment variable | Purpose |
|---|---|
| TOPOSPACE_API_KEY | Customer-facing bearer credential |
| TOPOSPACE_TENANT_ID | Tenant identifier |
| PRIMARY_PROVIDER_BASE_URL | OpenAI-compatible provider base URL |
| PRIMARY_PROVIDER_API_KEY | Upstream provider secret |
| FALLBACK_PROVIDER_* | Optional secondary provider |
| TOPOSPACE_RPM | Requests-per-minute quota |
| TOPOSPACE_MONTHLY_TOKEN_LIMIT | Monthly token quota |
| TOPOSPACE_MONTHLY_COST_LIMIT | Monthly cost quota |
| PROVIDER_RETRIES | Retry count |
| ALLOW_MOCK_PROVIDER | Development only; false in production |

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

Production secrets should be stored in Replit Secrets or another managed secret store. Temporary development URLs should not be treated as permanent customer endpoints.

# 15. Security and Multi-Tenancy

- Derive tenant identity from authenticated credentials rather than trusting arbitrary client tenant fields.
- Do not log raw customer API keys or provider credentials.
- Separate customer-facing API keys from upstream provider secrets.
- Use durable storage for tenant, key, usage, policy, and audit state.
- Use distributed rate limiting for multi-instance deployments.
- Keep deterministic security/policy controls outside learned optimization.
- Apply request/response redaction and privacy policy to telemetry.
- Maintain release manifests and SBOM evidence.

# 16. Validation Methodology

The packaged TopoSpace Cloud 1.3.0-beta1 validation reported 293 repository tests passing, together with Python compile checks and Replit deployment-asset checks. These tests validate software behavior and regression, not real-world commercial savings.

Before general availability, validation should additionally include real provider contract tests, concurrent load tests, streaming disconnect/cancellation tests, PostgreSQL/Redis failure scenarios, multi-tenant isolation tests, billing reconciliation, security review, and production A/B measurement.

# 17. Commercial Product Model

A practical commercial model is to offer TopoSpace as a cloud gateway with BYOK provider credentials initially, reducing provider-credit and accounting risk while allowing customers to retain their existing model contracts.

| Edition | Target | Commercial structure |
|---|---|---|
| Developer | Individual/small team | Subscription + usage |
| Growth | AI SaaS/product teams | Subscription + optimized request/usage |
| Enterprise | Large/regulatory organizations | Annual contract + SLA |
| Private / On-prem | Regulated/private infrastructure | License + support |

The principal product category should remain broader than a simple LLM gateway: AI Execution Optimization Plane or Adaptive Execution Middleware.

# 18. Roadmap

| Release | Primary objective |
|---|---|
| 1.3 beta1 | Replit-ready commercial gateway |
| 1.3 beta2 | Concrete PostgreSQL/Redis, durable key and usage state |
| 1.3 beta3 | Provider-specific streaming, cancellation, tool-call normalization |
| 1.3 RC | Customer/admin console, audit, pricing catalog, export |
| 1.3 GA | Load/security/SLA hardening and production benchmarks |
| 1.4 | TorusDB Agent Memory connector and richer topology/policy packs |

# 19. Limitations and Production Gaps

The current beta is a commercialization-ready foundation, not a claim that every production subsystem is complete. The following areas remain explicit hardening work:

- PostgreSQL system-of-record implementation and migrations.
- Redis-backed distributed rate limiting and coordination.
- Provider-specific upstream streaming and cancellation semantics.
- Full API-key lifecycle: issue, revoke, rotate, scope, audit.
- Invoice-grade immutable billing ledger and reconciliation.
- Centralized metrics, logs, traces, alerting, and SLOs.
- Security threat modeling, penetration testing, backup/restore, and operational runbooks.
- Live provider A/B evidence before publishing savings percentages.

# 20. Conclusion

TopoSpace reframes AI optimization from isolated model routing into a joint execution problem. By making context geometry, model selection, execution graph, and token budget part of one control plane, it provides a path toward more efficient and explainable agent infrastructure without forcing applications into a specific framework or provider.

The commercial direction is therefore not merely another AI SDK. It is a middleware and cloud control plane that can sit underneath AI applications, observe execution as a topology, optimize decisions under measurable objectives, and safely evolve policies through production operations.

**Author**
Caleb Lee
Email: director@j1729labs.online